

```
/**  
 * Prints out the TicTacToe board with all the values.  
 */  
public static void printBoard() {  
    "Hier code einfügen"  
}
```

ÜBUNG 6

24WS-UE06-A01 Feld initialisieren und ausgeben

Schreiben Sie eine Funktion `public static int[] getRandomArray(int size, int limit)`, die ein `int`-Array der Größe `size` erzeugt, mit Zufallszahlen zwischen 1 und `limit` befüllt, und das erstellte Array als Ergebnis zurückliefert. Die Funktion soll ein leeres Feld (Größe 0) als Ergebnis liefern, wenn `size` oder `limit` ≤ 0 sind.

Hinweis: Die Anweisung `int x = (int) (Math.random() * y) + 1` initialisiert `x` mit einem Wert zwischen 1 und `y`.

getRandomArray implementiert Keine Ergebnisse

Schreiben Sie weiters eine Methode `public static String toString(int[] array)`, die ein `int`-Array als Parameter erhält, und eine String-Repräsentation dieses Arrays zurückgibt. Die String-Repräsentation soll dabei folgendes Format haben:

- der Beginn des Arrays soll mit einer öffnenden Klammer (markiert werden
- das Ende des Arrays soll mit einer schließenden Klammer) markiert werden
- dazwischen sollen die einzelnen Werte des Arrays der Reihe nach ausgegeben werden
- die einzelnen Werte sollen dabei mit einem Strichpunkt (';') separiert werden
- Falls als Parameter `null` übergeben wird, soll anstatt eines Strings der Wert `null` zurückgegeben werden.

Beispiel-Resultat eines Feldes mit den Werten von 1-5: "(1; 2; 3; 4; 5)"

Testen Sie Ihre Implementierung der `toString`-Methode, indem Sie ein vordefiniertes Array übergeben, und das Ergebnis auf der Konsole ausgeben.

toString implementiert Keine Ergebnisse

Schreiben Sie außerdem eine Methode `public static void print(int[] array)`, welche die Methode `toString()` verwendet, um die String-Repräsentation des übergebenen Arrays auf der Konsole auszugeben.

24WS-UE06-A02 Rechnen mit Feldern

In dieser Aufgabe sollen Sie mit den Werten innerhalb von Feldern rechnen. Verwenden Sie zur Erstellung bzw. der Ausgabe der verwendeten Feldern innerhalb der main-Methode (Ausgabe zu Test/Debugging-Zwecken) die Methoden `public static int[] getRandomArray(int size, int limit)` und `public static String toString(int[] array)` aus Aufgabe 1 wieder.

Berechnungen auf 1 Feld

Schreiben Sie folgende Funktionen, welche Berechnungen mit den Elementen eines ganzzahligen Feldes durchführen:

- `public static int getSum(int[] array)`, `public static int getProduct(int[] array)` bildet die Summe bzw. das Produkt aller Feldelemente. Liefert -1 wenn array null oder ein leeres Feld ist.
- `public static double getDividedSum(int[] array, int number)` dividiert die Summe der ersten number Feldelemente durch die Summe der letzten number Feldelemente. Liefert -1 wenn array null oder ein leeres Feld ist, bzw. die Größe von array < 2 x number ist, bei der Berechnung eine Division durch 0 passieren würde.
- `public static double getAverage(int[] array)`, `public static int getMin(int[] array)`, `public static int getMax(int[] array)` berechnet den Mittelwert, das Minimum und das Maximum der Feldelemente. Liefert -1 wenn array null oder ein leeres Feld ist.

Berechnungen auf 2 Feldern

- `public static int[] add(int[] array1, int[] array2)` erstellt und retourniert ein neues Feld, welches an jeder Position die Summe der jeweiligen Feldposition der übergebenen Felder (array1 und array2) beinhaltet. Liefert null zurück, wenn array1 und array2 nicht die selbe Länge haben, oder einer dieser Parameter null ist.
- `public static int[] subtract(int[] array1, int[] array2)` erstellt und retourniert ein neues Feld, welches an jeder Position die Differenz der jeweiligen Feldposition der übergebenen Felder (Hinweis: $\text{array1}[i] - \text{array2}[i]$) beinhaltet. Liefert null zurück, wenn array1 und array2 nicht die selbe Länge haben, oder einer dieser Parameter null ist.
- `public static int[] multiply(int[] array1, int[] array2)` erstellt und retourniert ein neues Feld, welches an jeder Position das Multiplikationsergebnis der jeweiligen Feldposition der übergebenen Felder (array1 und array2) beinhaltet. Liefert null zurück, wenn array1 und array2 nicht die selbe Länge haben, oder einer dieser Parameter null ist.
- `public static int[] divide(int[] array1, int[] array2)` erstellt und retourniert ein neues Feld, welches an jeder Position das Divisionsergebnis der jeweiligen Feldposition der übergebenen Felder (Hinweis: $\text{array1}[i] / \text{array2}[i]$) beinhaltet. Liefert null zurück, wenn array1 und array2 nicht die selbe Länge haben, einer dieser Parameter null ist, oder ein Wert im Feld array2 == 0 ist (Division durch 0 nicht möglich).

24WS-UE06-A03 Feld invertieren

Schreiben Sie eine Funktion `public static int[] invert(int[] a)`, die ein Feld von ganzen Zahlen als Parameter erhält und ein neu erzeugtes Feld mit invertiertem Inhalt als Ergebnis liefert. Achten Sie auf eine sinnvolle Behandlung von Sonderfällen! Hinweise: Die Invertierung von null ergibt null, die Invertierung eines leeren Feldes ergibt ein leeres Feld. Schreiben Sie außerdem eine Funktion `public static boolean isInverted(int[] a, int[] b)`, welche überprüft, ob die beiden übergebenen Felder `a` und `b` jeweils invertierte Felder sind.

Verwenden Sie wie im Beispiel unten Methoden der vorherigen Aufgaben zum Testen Ihrer Implementierung! Geben Sie dabei zum Testen der Methode `invert` (wie im Beispiel unten) zuerst das generierte Array aus, und anschließend das invertierte Array, um die Ergebnisse vergleichen zu können. Falls Ihr Programm keine Ausgabe erzeugt, bzw. nicht die Methoden von Aufgabe 1 wiederverwendet, werden Ihnen Punkte abgezogen.

Beispiel Aufruf:

```
int[] a = getRandomArray(5,10);
int[] b = invert(a);
Out.println(toString(a));
Out.println(toString(b));
a = getRandomArray(10,100);
b = invert(a);
Out.println(toString(a));
Out.println(toString(b));
```

Beispiel Ausgabe:

```
[7; 10; 10; 1; 6]
[6; 1; 10; 10; 7]
[69; 83; 8; 26; 53; 46; 37; 42; 98; 84]
[84; 98; 42; 37; 46; 53; 26; 8; 83; 69]
```

24WS-UE06-A04 Sortierung von Feldern

Schreiben Sie eine Funktion `public static boolean isSorted(int[] field, boolean isDescending)`, die überprüft, ob ein Feld aus ganzzahligen Werten aufsteigend bzw. absteigend sortiert ist. Hat der Parameter `isDescending` den Wert `true`, dann soll überprüft werden, ob das Feld absteigend sortiert ist. Hat `isDescending` den Wert `false`, dann soll überprüft werden, ob das Feld aufsteigend sortiert ist. Ist `field` gleich `null` oder die Feldlänge 0, dann soll die Funktion `true` liefern.

Beispiel-Content Main-Methode:

```
final boolean ASCENDING = false;
final boolean DESCENDING = true;
int[] a= null;
Out.println(toString(a));
Out.println("Aufsteigend sortiert: " + isSorted(a, ASCENDING));
Out.println("Absteigend sortiert: " + isSorted(a, DESCENDING));
a= new int[]{1,2,3,4,5,6,7,8,9,0};
Out.println(toString(a));
Out.println("Aufsteigend sortiert: " + isSorted(a, ASCENDING));
Out.println("Absteigend sortiert: " + isSorted(a, DESCENDING));
a= new int[]{1,2,3,4,5,6,7,8,9,100};
Out.println(toString(a));
Out.println("Aufsteigend sortiert: " + isSorted(a, ASCENDING));
Out.println("Absteigend sortiert: " + isSorted(a, DESCENDING));
a= new int[]{3,2,2,1};
Out.println(toString(a));
Out.println("Aufsteigend sortiert: " + isSorted(a, ASCENDING));
Out.println("Absteigend sortiert: " + isSorted(a, DESCENDING));
```

Beispiel Ausgabe:

```
null
Aufsteigend sortiert: true
Absteigend sortiert: true
[1, 2, 3, 4, 5, 6, 7, 8, 9, 0]
Aufsteigend sortiert: false
Absteigend sortiert: false
[1, 2, 3, 4, 5, 6, 7, 8, 9, 100]
```

Aufsteigend sortiert: true

Absteigend sortiert: false

[3, 2, 2, 1]

Aufsteigend sortiert: false

Absteigend sortiert: true

24WS-UE06-A05 ConnectFour

Lesen Sie den gesamten nachfolgenden Text bevor Sie mit Ihrer Lösung starten!

Schreiben sie ein Java-Programm, welches eine 4-Gewinnt Spiel ([Siehe Link](#)) repräsentiert. 4-Gewinnt ist ein Strategiespiel für zwei Spieler, das auf einem Spielfeld bestehend aus *sieben Spalten und sechs Zeilen* gespielt wird. Die Spieler*innen setzen abwechselnd ihre Steine in die Spalten. Ziel ist es, vier gleiche Steine in einer horizontalen, vertikalen oder diagonalen Linie zu platzieren.

Aufgabenstellung

Es soll ausgegeben werden, welcher Spieler*in an der Reihe ist ("Player X" oder "Player O"). Anschließend soll die Spaltenauswahl von der Spieler*in eingegeben werden. Diese geben an, wo der Kreis oder das Kreuz am Spielfeld platziert werden sollen.

Die eingegebene Ziffer gibt die Spalte an, in der das Zeichen aufgesetzt wird. Beispielsweise führt die Konsoleneingabe 3 dazu, dass das von unten erste freie Feld in der dritten Spalte befüllt wird.

Weiterhin soll die Spalteneingabe einer Validierung unterzogen werden. Dabei sollen die folgenden Fälle überprüft werden:

- Die eingegebene Spaltennummer darf nicht außerhalb des Spielfeldbereichs liegen.
- Bereits volle Spalten dürfen nicht mehr gewählt werden.

Das Spiel soll mit einer Erfolgsmeldung beendet werden, wenn ein Spieler*in vier gleiche Zeichen vertikal oder horizontal platzieren konnte. **Eine Überprüfung der diagonalen Gewinnbedingung ist für diese Aufgabe nicht notwendig!** Im Falle von Unentschieden (alle Felder befüllt, allerdings kein Gewinner*in) soll ebenfalls eine entsprechend Ausgabe auf der Konsole ausgegeben werden.

Anstatt die gesamte Funktionalität in der main-Funktion zu deklarieren, sollen separate Funktionen erstellt werden. Nutzen Sie dazu die vorgefertigten Methodenköpfe und Variablen.

Nutzen sie für die Darstellung der sechs Zeilen Felder (Arrays). Die Nummerierung der Reihen ist dabei von oben nach unten. Das heißt, `public static char[] row1` repräsentiert die oberste Zeile im 4-Gewinnt Spielfeld usw.

Ein typischer Spielverlauf könnte entsprechend wie folgt aussehen:

```
+---+---+---+---+---+---+
| | | | | | |
+---+---+---+---+---+---+
| | | | | | |
```

```
+---+---+---+---+---+---+
| | | | | | |
+---+---+---+---+---+---+
| | | | | | |
+---+---+---+---+---+---+
| | | | | | |
+---+---+---+---+---+---+
| | | | | | |
+---+---+---+---+---+---+
| | | | | | |
+---+---+---+---+---+---+
1 2 3 4 5 6 7
```

Player X: 3

```
+---+---+---+---+---+---+
| | | | | | |
+---+---+---+---+---+---+
| | | | | | |
+---+---+---+---+---+---+
| | | | | | |
+---+---+---+---+---+---+
| | | | | | |
+---+---+---+---+---+---+
| | | | | | |
+---+---+---+---+---+---+
| | | | | | |
+---+---+---+---+---+---+
| | |X| | | |
+---+---+---+---+---+---+
1 2 3 4 5 6 7
```

Player O: 3

```
+---+---+---+---+---+---+
| | | | | | |
+---+---+---+---+---+---+
| | | | | | |
+---+---+---+---+---+---+
| | | | | | |
+---+---+---+---+---+---+
| | | | | | |
```



```

+---+---+---+---+---+---+
| | |O| | | | |
+---+---+---+---+---+
| | |X| | | | |
+---+---+---+---+---+
1 2 3 4 5 6 7

```

Player X: 4

```

+---+---+---+---+---+---+
| | | | | | | |
+---+---+---+---+---+
| | | | | | | |
+---+---+---+---+---+
| | | | | | | |
+---+---+---+---+---+
| | | | | | | |
+---+---+---+---+---+
| | | | | | | |
+---+---+---+---+---+
| | |O| | | | |
+---+---+---+---+---+
| | |X|X| | | | |
+---+---+---+---+---+
1 2 3 4 5 6 7

```

Player O: 3

```

+---+---+---+---+---+---+
| | | | | | | |
+---+---+---+---+---+
| | | | | | | |
+---+---+---+---+---+
| | | | | | | |
+---+---+---+---+---+
| | |O| | | | |
+---+---+---+---+---+
| | |O| | | | |
+---+---+---+---+---+
| | |X|X| | | | |

```

```

+---+---+---+---+---+---+
1 2 3 4 5 6 7

```

Player X: 5

```

+---+---+---+---+---+---+
| | | | | | |
+---+---+---+---+---+---+
| | | | | | |
+---+---+---+---+---+---+
| | | | | | |
+---+---+---+---+---+---+
| | | | | | |
+---+---+---+---+---+---+
| | | O | | | |
+---+---+---+---+---+---+
| | | O | | | |
+---+---+---+---+---+---+
| | | X | X | X | | |
+---+---+---+---+---+---+
1 2 3 4 5 6 7

```

Player O: 2

```

+---+---+---+---+---+---+
| | | | | | |
+---+---+---+---+---+---+
| | | | | | |
+---+---+---+---+---+---+
| | | | | | |
+---+---+---+---+---+---+
| | | O | | | |
+---+---+---+---+---+---+
| | | O | | | |
+---+---+---+---+---+---+
| | O | X | X | X | | |
+---+---+---+---+---+---+
1 2 3 4 5 6 7

```

Player X: 6

```

+---+---+---+---+---+---+
| | | | | | |
+---+---+---+---+---+---+
| | | | | | |
+---+---+---+---+---+---+
| | | | | | |
+---+---+---+---+---+---+
| | | O | | | |
+---+---+---+---+---+---+
| | | O | | | |
+---+---+---+---+---+---+
| | O | X | X | X | X |
+---+---+---+---+---+---+
  1  2  3  4  5  6  7

```

Congrats, X won!

-- END --

Process finished with exit code 0

Methodenköpfe und Variablen

```

// utility vars
public static final char EMPTY_CHAR = ' ';
public static final char X_PLAYER = 'X';
public static final char O_PLAYER = 'O';
public static final String ROW_DELIMITER = " +---+---+---+---+---+---+ ";
public static final String COL_DELIMITER = " | ";

public static char[] row1 = new char[7];
public static char[] row2 = new char[7];
public static char[] row3 = new char[7];
public static char[] row4 = new char[7];
public static char[] row5 = new char[7];
public static char[] row6 = new char[7];

```

```
public static char currentPlayer = X_PLAYER;

public static void main(String[] args) {
    "Hier Code einfügen"
}

/**
 * Fills an entire array with the empty character and returns the array.
 */
public static char[] fillArrayEmptyChar(char[] arr) {
    "Hier Code einfügen"
}

/**
 * Places the currentPlayer character into the respective column on the ConnectFour grid.
 */
public static void makeMove(int userColumn) {
    "Hier Code einfügen"
}

/**
 * Switches the currentPlayer variable to X, if O is currently playing and vice versa.
 */
public static void switchPlayer() {
    "Hier Code einfügen"
}

/**
 * Checks, whether the ConnectFour game board is filled entirely.
 * If it is full, this function returns true.
 */
public static boolean isBoardFull() {
    "Hier Code einfügen"
}
```

```
/**
 * Checks, if the column specified by a user input is already full.
 * Method returns true, if the column is full.
 */
public static boolean isColumnFull(int userColumn) {
    "Hier Code einfügen"
}

/**
 * Checks, whether the user input column is on the board.
 * Returns true, if the column is in range.
 */
public static boolean isMoveInRange(int userColumn) {
    "Hier Code einfügen"
}

/**
 * Prints out the entire game board.
 */
public static void printBoard() {
    "Hier Code einfügen"
}

/**
 * Checks, whether a player won and returns the winning character (PLAYER_X or PLAYER_O).
 * In case neither of them won, the empty character shall be returned.
 */
public static char getWinner() {
    "Hier Code einfügen"
}
```